



考前重点:

Chapter 6 — Synchronized Tools

- ✓ - Race Condition 名词解释、举例
- ✓ - Critical Section 名词解释
 - Peterson's solution
- ✓ - test_and_set
- ✓ - Spin Lock
- ✓ - Semaphore

P. 7 ~ 9

P. 21 (Synchronized)

P. 26 ~ 31

P. 37 ~ 38

P. 49 ~ 51

P. 53 ~ 58

Chapter 8 — DeadLocks

- D.L. 要件 & 特徵
- ✓ - D.L. Prevention

P. 8 ~ 9

P. 20 ~ 26

Chapter 9 — Main Memory

- ✓ - Contiguous Memory Allocation
- ✓ - External Fragmentation
- ✓ - Internal Fragmentation

P. 23 ~ 27

P. 29

P. 32 ~ 33

Chapter 10 — Virtual Memory

- Page Fault
- Page Replacement Concept
- ✓ (- FIFO、Optimal、LRU
- Belady's anomaly

P. 9

P. 12 ~ P. 13

P. 19、P. 25、P. 27

P. 23

1. 我考的是 ____ 卷。
2. 請說明何謂 Race Condition (5%)，並嘗試舉一個例子說明 (5%)。
3. 請說明何謂 Critical Section (10%)。
4. 使用 test_and_set() 寫出符合 Bounded Waiting 和 Mutual Exclusion 要求，進入 Critical Section 的 Pseudo Code (10%)。test_and_set() 的定義如下：

```
boolean test_and_set(boolean *target) {  
    boolean rv = *target;  
    *target = true;  
  
    return rv;  
}
```

5. 請說明 Spinlock (busy waiting) 的缺點接著說明 Semaphore (go to sleep) 如何避免此缺點 (5%)。並說明他們各自的使用時機 (5%)。
6. 請說明死結防止 (Deadlock Prevention) 方法中，如何讓 Circular Wait 條件不成立的方法 (10%)。
7. 請解釋何謂 External Fragmentation (5%)；何謂 Internal Fragmentation (5%)
8. 請說明 Page Size 太大或太小的優缺點 (10%)。
9. Given five memory partitions of 170 Kb, 520 Kb, 270 Kb, 310 Kb, 620 Kb (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of 220 Kb, 390 Kb, 130 Kb, and 450 Kb (in order) (15%)。
10. 假設記憶體共有 3 個 Frame，而 Memory Reference String 為 6, 0, 1, 2, 0, 3, 0, 5, 1, 0, 3, 1, 2, 0, 2, 0, 1, 6, 0, 1，試寫出 FIFO Page Replacement, Optimal Page Replacement 和 LRU Page Replacement 的結果，並計算發生多少次 Page Fault (15%)。

2、

多个 Process / Thread 同時存取並修改同一份資料，且沒有同步機制時，會導致最後的執行結果依賴實際執行的先後順序，而不是正確的預期結果。

e.g.

一變數 $x=3$ ，兩 Process A、B 同時修改變數 x ：

A: $x = x + 1$
B: $x = x + 2$ $\rightarrow$ 若沒有適當的同步機制，可能會因 context switch 導致結果不為預期的 6。

3、

Critical Section 是一段「受保護」的程式碼，在同一時間中，只會有 1 個 Process / thread 進入這段受保護的區域，來避免 race condition。

4、

```
while (True)
    waiting[i] = True;
    while (waiting[i] and test_and_set(&lock));
    waiting[i] = false;
```

檢查是否可進入 C.S.，並設定鎖的狀態。

```
// Critical Section
```

```
j = (i + 1) % N;
```

```
while (j != i and !waiting[j]) j = (j + 1) % N;
```

找下一個想進 C.S. 的人

```
if (j == i) lock = 0;
```

$j = i$ ，找了一輪找回自己（表沒有人想進 C.S.），把鎖打開。

```
else waiting[j] = false;
```

發現還有人再等，鎖不放，直接交棒。

```
}
```

5、

spin lock 的實作方法是讓 CPU「忙碌」的執行 while loop 檢查，浪費 CPU 資源。

→ 適合用在 Critical Section 短、預期很快拿到鎖的情況。

Semaphore 與 spin lock 不同之處在於，實作方法是將還拿不到鎖的 Process 丟入 waiting queue，讓 CPU 先去執行別的 Process，大大降低 CPU 負擔。

→ 適合用在 Critical Section 長需要等很久，或 I/O Event。

6、

將所有的資源都編號，並規定所有 Process 在拿資源時只能請求比現有資源編號還高的資源。不會有程式回頭要低編號的資源，循環等待鏈就無法成形。

7、

External Fragmentation: 空間的記憶體空間被切分成很多不連續的小洞，雖然總體剩餘可用足夠，卻因不連續，導致無法容納下新的 Process。

Internal Fragmentation: 非所有 Process 都是 Pages 的整數倍，為了補足 Page Size，會讓 Process 補到 Page Size 大小的整數倍，致 block 內多補的空白空間被浪費。

8、

Page Size 太大:

advantage: page table 較小, 減少額外開銷, 降低管理難度。

disadvantage: internal fragmentation 影響更嚴重。Page Fault 的代價更大。

Page Size 太小:

advantage: internal fragmentation 較小, 浪費較少空間。

disadvantage: page table 變大, 管理難度變高。

9、

process: 220kb 390kb 130kb 450kb (in order)

mem partitions: 170kb 520kb 270kb 310kb 620kb (in order)

First Fit:

220kb → 520kb (170kb, 300kb, 270kb, 310kb, 620kb)

390kb → 620kb (170kb, 300kb, 270kb, 310kb, 230kb)

130kb → 170kb (40kb, 300kb, 270kb, 310kb, 230kb)

450kb → X external fragmentation.

9.

process: 220kb 390kb 130kb 450kb (in order)

mem partitions: 170kb 520kb 270kb 310kb 620kb (in order)

Best-Fit:

220kb → 270kb (170kb, 520kb, 50kb, 310kb, 620kb)

390kb → 520kb (170kb, 130kb, 50kb, 310kb, 620kb)

130kb → 130kb (170kb, 0kb, 50kb, 310kb, 620kb)

450kb → 620kb (170kb, 0kb, 50kb, 310kb, 170kb)

Worst Fit:

220kb → 620kb (170kb, 520kb, 270kb, 310kb, 400kb)

390kb → 520kb (170kb, 130kb, 270kb, 310kb, 400kb)

130kb → 400kb (170kb, 130kb, 270kb, 310kb, 270kb)

450kb → X external fragmentation.

10.

mem reference string: 6 0 1 2 0 3 0 5 1 0

* 3 Frame 3 1 2 0 2 0 1 6 0 1

FIFO: 先進先出

6 0 1 2 0 3 0 5 1 0 3 1 2 0 2 0 1 6 0 1

6 6 ~~2~~ 2 2 2 ~~5~~ 5 5 5 5 ~~2~~ 2 2 2 2 ~~6~~ 6 6 6

0 0 0 ~~3~~ 3 3 ~~1~~ 1 1 1 1 ~~0~~ 0 0 0 0 0 0 0

1 1 1 ~~0~~ 0 0 0 ~~3~~ 3 3 3 3 3 ~~1~~ 1 1 1 1

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Page fault: 13 次

Optimal: 拿掉最晚用到的!

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
6	0	1	2	0	3	0	5	1	0	3	1	2	0	2	0	1	6	0	1

6	6	2	2	3	5	5	5	3	3	2	2	2	2	2	6	6	6		
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Page Fault: 9次

LRU: 最近最少被用到的

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
6	0	1	2	0	3	0	5	1	0	3	1	2	0	2	0	1	6	0	1

6	6	2	2	2	2	5	5	5	3	3	3	0	0	0	0	6	0	0	
	0	0	0	0	0	0	0	0	0	0	2	2	2	2	2	6	6	6	
		1	1	3	3	3	1	1	1	1	1	1	1	1	1	1	1	1	1

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Page Fault = 11次

Peterson's solution :

```
while (True){
```

```
    flag[i] = true;
```

← 進 C.S. 前把 self 設成 True

```
    turn = j;
```

← turn 設給別人

```
    while (flag[j] and turn == j):
```

← while wait

```
        // Critical Section
```

```
        flag[i] = false
```

← left C.S.

```
}
```

→ 基於軟體來實現。

→ 在操作 flag、turn 導致 Context Switch 會有問題，現代計算機不一定能用。

→ 現代 Computer 優化可能會 reorder

Dead Lock 達成條件：

1. Mutual exclusion 互斥、都想建立 C.S.

2. Hold and wait 持有一個資源、同時等待另一個資源

3. No preemption 非搶佔式、一個 Process 不能搶另一個 Process 的資源。

4. Circular wait 循環等待、每個 Process 都在等另一個 Thread

Page Fault

當 Program 嘗試訪問一個尚未載入進實體記憶體，仍然存在 Virtual Memory 的 frame 時，就會觸發 Page Fault，將該 Frame 從 Virtual Memory 載入到實體記憶體。

Belady's anomaly

在某些 Page 置換算法(如 FIFO)中，把可用 frame 數增加時，Page Fault 發生次數可能不減反增。